1. Smart Home Thermostat Controller

Scenario: You are programming an automated smart thermostat. The system adjusts the room temperature based on the time of day (`"morning"`, `"afternoon"`, `"evening"`, `"night"`), whether the house is occupied (`True`/`False`), and the current season (`"summer"`, `"winter"`).
Task: Write a program that takes these three inputs and determines the target temperature:

- If the house is unoccupied, set the temperature to `$18^\circ\text{C}$` in winter and `$28^\circ\text{C}$` in summer to save energy.
- If occupied during winter: set to `$22^\circ\text{C}$` in the morning/evening, `$20^\circ\text{C}$` in the afternoon, and `$19^\circ\text{C}$` at night.
- If occupied during summer: set to `$23^\circ\text{C}$` during the day (morning/afternoon/evening) and `$21^\circ\text{C}$` at night.

2. ATM Cash Dispenser with Note Availability

Scenario: An ATM has a limited number of currency notes: 10 notes of `$\$50$`, 20 notes of `$\$20$`, and 30 notes of `$\$10$`.
Task: Write a `while` loop simulation that prompts a user for a withdrawal amount.

- Validate that the amount is a multiple of 10 and does not exceed the total cash available in the ATM.
- Dispense the cash using the highest denominations possible first, updating the remaining note counts after each transaction.
- The loop should terminate when the ATM runs out of money or the user enters `0` to exit.

3. Warehouse Inventory Reorder Alert System

Scenario: A warehouse tracks its daily inventory changes over a 7-day period. If an item's stock drops below the reorder threshold (20 units), an urgent restock order of 50 units is triggered at the end of that day (arriving the next morning). However, if the stock hits 0 during the day, any excess demand is recorded as "lost sales."
Task: Given an initial stock of 45 units and a list of daily customer demands `demands = [12, 18, 25, 10, 30, 22, 15]`, write a `for` loop to simulate the week. Print the day number, starting stock, ending stock, whether a restock was triggered, and total lost sales at the end of the week.

4. Server Log Anomaly Detector

Scenario: You are monitoring server response times (in milliseconds) streamed in a list: `logs = [120, 135, 110, 520, 580, 610, 140, 125, 900, 115]`. An anomaly alert is triggered only when **three consecutive** response times exceed 500 ms.
Task: Write a loop to iterate through the log data. If three consecutive spikes are detected, print the exact index range where the anomaly occurred and halt further scanning using `break`. If the loop completes without finding three consecutive spikes, print `"System Stable"`.

5. Subscription Tier Upgrade Analyzer

Scenario: A SaaS company tracks monthly active usage hours for a list of users. Users are on the `"Basic"` tier (10 hours limit). Every hour over the limit costs $\$2.50$. If a user's overage charges exceed the cost of the `"Pro"` tier upgrade ($\$15$ difference) for two consecutive months, the system recommends an upgrade.
Task: Given a dictionary where keys are usernames and values are lists of 6 monthly usage hours, use nested loops and `if-else` conditions to generate a list of users who should receive an upgrade recommendation.

6. E-Commerce Dynamic Pricing Engine

Scenario: An online store calculates the final price of a cart based on multiple optional factors.
Task: Write a function `calculate_checkout(cart_total, tax_rate=0.08, discount_code=None, is_member=False)`:

- Apply a 10% discount if `discount_code == "SAVE10"`, or a flat $\$20$ off if `discount_code == "FLAT20"` (only valid if `cart_total >= 100`).
- If `is_member` is `True`, apply an additional 5% discount on the discounted subtotal.
- Apply the `tax_rate` at the very end. Return the final rounded price.

7. Automated Email Retry Mechanism

Scenario: When sending automated reports, network glitches can cause failures.
Task: Create a function `connect_to_server(server_ip, port=8080, max_retries=3, timeout=5.0)`. Simulate a connection attempt inside the function (you can pass a dummy list of boolean statuses like `[False, False, True]`). The function should loop up to `max_retries` times. If it connects, return `"Connected to <server_ip>:<port>"`. If all retries fail, return `"Connection Timed Out after <max_retries * timeout> seconds"`.

8. Loan EMI & Amortization Calculator

Scenario: A fintech app needs a reusable function to calculate Equated Monthly Installments (EMI).
Task: Write a function `calculate_emi(principal, annual_rate=8.5, tenure_years=5, processing_fee_pct=1.0)`.

- Calculate the monthly interest rate $r = \frac{\text{annual_rate}}{12 \times 100}$ and total months $n = \text{tenure_years} \times 12$.
- Use the formula: $$\text{EMI} = P \times r \times \frac{(1 + r)^n}{(1 + r)^n - 1}$$
- Return a dictionary containing `monthly_emi`, `total_payable`, and `processing_fee`.

9. Text Sanitizer for NLP Pipeline

Scenario: Before feeding customer reviews into a machine learning model, the text must be cleaned based on configurable rules.
Task: Define a function `sanitize_text(text, lowercase=True, remove_punctuation=True, stop_words=None)`.

- If `stop_words` is `None`, default it inside the function to `["the", "is", "at", "which", "on"]`.
- Process the string according to the boolean flags, remove any words present in `stop_words`, and return the cleaned string.

10. Multi-Branch Sales Aggregator (`*args`)

Scenario: A retail chain has a varying number of pop-up stores reporting daily revenue. Some days 3 stores report; other days 10 stores report.
Task: Write a function `analyze_daily_sales(target_revenue, *branch_sales)` that accepts a daily target followed by any number of branch revenue figures. The function should calculate the total revenue, average branch revenue, the number of branches that beat the average, and whether the overall `target_revenue` was met.

11. Dynamic SQL Query Builder (`**kwargs`)

Scenario: A backend developer needs a helper function to generate `SELECT` SQL queries dynamically based on user-applied filters on a dashboard.
Task: Write a function `build_query(table_name, **filters)` that takes a table name and arbitrary keyword arguments representing column-value pairs (e.g., `status="active"`, `role="admin"`, `min_age=25`).

- Return a formatted SQL string: `"SELECT * FROM users WHERE status='active' AND role='admin' AND min_age=25;"`.

- If no `**filters` are passed, return `"SELECT * FROM <table_name>;"`.

12. Hospital Patient Triage Logger (`*args` & `**kwargs`)

Scenario: An emergency room intake system records a patient's name, an arbitrary number of reported symptoms, and optional vital signs/metadata (like `blood_pressure`, `heart_rate`, `allergies`).
Task: Write a function `register_patient(patient_name, *symptoms, critical_flag=False, **vitals)`.

- Format and print a clean medical intake report.
- Automatically set `critical_flag` to `True` in the output if `"chest pain"` or `"shortness of breath"` is in `*symptoms`, or if `heart_rate > 120` is passed in `**vitals`.

13. Smart Data Pipeline Router (`*args` & `**kwargs`)

Scenario: A data engineering pipeline takes multiple raw numerical datasets (lists) and applies configurable transformation rules passed as keyword arguments.
Task: Write a function `process_datasets(*datasets, **operations)`.

- Combine all lists passed in `*datasets` into a single flattened list.
- Check `**operations` for keys: if `drop_negatives=True`, remove numbers < 0; if `scale_factor` is provided, multiply all elements by that factor; if `offset` is provided, add that value to all elements. Return the processed list.

14. Restaurant Group Bill Splitter

Scenario: Friends dining at a restaurant order shared appetizers (`*shared_items` prices) and individual mains (`**individual_orders` where key is the person's name and value is their main course price).
Task: Write a function `split_bill(tax_pct=5.0, tip_pct=10.0, *shared_items, **individual_orders)`.

- Divide the total cost of `*shared_items` equally among all people present in `**individual_orders`.
- Add each person's individual order cost, then apply `tax_pct` and `tip_pct` proportionally to each person's share.
- Return a dictionary mapping each person's name to their final payable amount.

15. Flight Search Sorting Engine

Scenario: A travel aggregator fetches a list of flight dictionaries:

```python
flights = [
    {"airline": "AirMax", "price": 350, "duration_hrs": 4.5, "stops": 1},
    {"airline": "SkyJet", "price": 420, "duration_hrs": 2.5, "stops": 0},
    {"airline": "CloudNine", "price": 350, "duration_hrs": 3.8, "stops": 1},
    {"airline": "BudgetFly", "price": 290, "duration_hrs": 7.0, "stops": 2}
]
```

Task: Use `sorted()` with `lambda` functions to generate three separate sorted lists:

1. Sorted by `price` ascending, and if prices are tied, by `duration_hrs` ascending.
2. Sorted by a custom "convenience score" calculated as $\text{price} + (\text{duration_hrs} \times 30) + (\text{stops} \times 50)$.

16. Payroll Tax & Bonus Processor (`map` & `filter` with `lambda`)

Scenario: HR has a list of employee tuples: `employees = [("Alice", 85000, 4.8), ("Bob", 52000, 3.2), ("Charlie", 95000, 4.9), ("Diana", 60000, 3.9)]`, representing `(name, salary, performance_rating)`.
Task:

1. Use `filter()` and a `lambda` function to extract only employees eligible for a bonus (`performance_rating >= 4.0`).
2. Chain or pass that result into `map()` with a `lambda` function to grant them a 12% salary hike, returning a list of tuples `(name, new_salary)`.

17. IoT Sensor Calibration Factory (Closures & Lambdas)

Scenario: Different temperature sensors in a factory have slight hardware drift and require custom linear calibration using the formula $y = mx + c$, where m is the scale factor and c is the offset.
Task: Write a function `create_calibrator(scale=1.0, offset=0.0)` that returns a `lambda` function taking a raw sensor reading x and returning the calibrated reading. Demonstrate this by creating two calibrators (`sensor_A` and `sensor_B`) and applying them to a list of raw readings.

18. Freelance Invoice Generator

Scenario: A freelancer bills clients for fixed-price milestones, hourly tasks, and optional rush fees or discounts.
Task: Write a function `generate_invoice(client_name, hourly_rate=50, *milestone_payments, **hourly_tasks_and_modifiers)`:

- `*milestone_payments` contains flat amounts for completed phases.

- `**hourly_tasks_and_modifiers` contains task names mapped to hours worked (e.g., `ui_design=12`, `backend_api=20`), along with optional special keys like `rush_delivery=True` (adds 20% to total) and `loyalty_discount=0.10` (subtracts 10%).
- Separate the task hours from the modifier flags using loops and `if-else`, use a `lambda` to compute hourly costs, and return a detailed invoice summary dictionary.

19. Automated Grading & Curve System

Scenario: A university professor wants to grade students dynamically.
Task: Write a function `grade_students(passing_mark=50, curve_fn=None, *bonus_points, **student_scores)`:

- `**student_scores` holds student names and their raw exam scores (e.g., `John=42, Emma=88`).
- `*bonus_points` is a variable list of extra credit points awarded to the whole class (sum them up and add to every student's score).
- `curve_fn` is an optional `lambda` function (defaulting to `lambda x: x` if `None` is provided) applied to each student's score after bonus points (e.g., `lambda x: min(100, x * 1.05)`).
- Use a loop and `if-elif-else` to assign letter grades (`A`, `B`, `C`, `F` based on `passing_mark`) and return a dictionary of final results.

20. Rule-Based Fraud Detection Engine

Scenario: A bank processes a batch of transactions and evaluates them against a dynamic set of fraud-detection rules (lambda functions).
Task: Write a function `evaluate_transactions(transactions, risk_threshold=2, *fraud_rules, **account_settings)`:

- `transactions` is a list of dictionaries (each having `amount`, `country`, `hour`).
- `*fraud_rules` is a variable number of `lambda` functions passed as arguments. Each `lambda` takes a transaction dict and `account_settings` dict, returning `True` if suspicious (e.g., `lambda tx, acc: tx['amount'] > acc.get('max_limit', 5000)`).
- Loop through each transaction and evaluate it against all `*fraud_rules`. If the number of triggered rules is $\ge \text{risk_threshold}$, flag the transaction as `"BLOCKED"`, otherwise `"APPROVED"`. Return the annotated transaction list.